Software Testing Report (IEEE 829 standard)

Contents

Executive Summary:

This report confirms that the BankCore Enterprise Management System has been thoroughly tested through an extensive, and detailed testing cycle which included both manual and automated testing. All of the BankCore Enterprise Management System's modules where tested through multiple different tests to ensure that the application meets business requirements, and that it is ready for deployment.

During the testing phase, multiple tests where conducted which highlighted 17 bugs, which impacted the programs readiness for deployment. It should also be noted that these defects have been rectified to ensure that the BankCore Enterprise Management System meets business requirements, and that it is ready to be deployed.

Test Scope:

During the testing phase of the BankCore Enterprise Management System, the following where tested:

- Account Management module
- Transaction Engine module
- Interest Calculator module
- Loan Processing module
- Reporting Engine module
- Authentication module
- Validation Library module

It should also be noted that the following aspects where not tested:

- Front end UI
- External database connections
- Third-Party API integrations

Test Environment:

Testing workstations:

- Main Testing Workstation:
    - CPU: AMD Ryzen 7 5800X3D
    - GPU: AMD Radeon RX9060XT
    - RAM: 32GB RAM
    - ROM: 1TB Nvme M.3 SSD
    - OS: Windows 11 Pro 25H2
    - Tools:
        - .NET SDK 8.0
    - IDE: Visual Studio 2026
- Secondary Testing Workstation:
    - CPU: AMD Ryzen 5 5500
    - GPU: NVIDIA GeForce RTX 3050
    - RAM: 16GB RAM
    - ROM: 1TB Nvme M.3 SSD
    - OS: Windows 11 Pro 25H2

    - Tools:
        - .NET SDK 8.0
    - IDE: Visual Studio 2026

Other tools:

- NuGet Packages:
    - Microsoft.NET.Test.sdk
    - MSTest
    - NUnit
    - xUnit
    - Moq
    - FluentAssertions
    - Coverlet.Collector

Test Execution Summary:

Module	Run #	Date	Total TCs	Passed	Failed	Blocked	Not Run	Pass Rate
Account Management	Run 1	06/07/2026	46	43	3	0	0	93,48%
Authentication	Run 1	06/07/2026	32	29	2	0	1	90,63%
Validation Library	Run 1	06/07/2026	48	43	5	0	0	89,58%
Transaction Engine	Run 1	06/07/2026	60	57	3	0	0	95,00%
Reporting Engine	Run 1	06/07/2026	10	10	0	0	0	100,00%
Interest Calculator	Run 1	06/07/2026	62	47	15	0	0	75,81%
Loan Processing	Run 1	06/07/2026	23	20	3	0	0	86,96%

OVERALL TEST EXECUTION SUMMARY							
Run #	Date	Total TCs	Passed	Failed	Blocked	Not Run	Pass Rate
Run 1 Baseline	06/07/2026	281	249	31	0	1	88,61%
Run 2 After Fix 1	07/07/2026	31	31	0	0	0	100,00%
Run 3 Regression	08/07/2026	115	115	0	0	0	100,00%

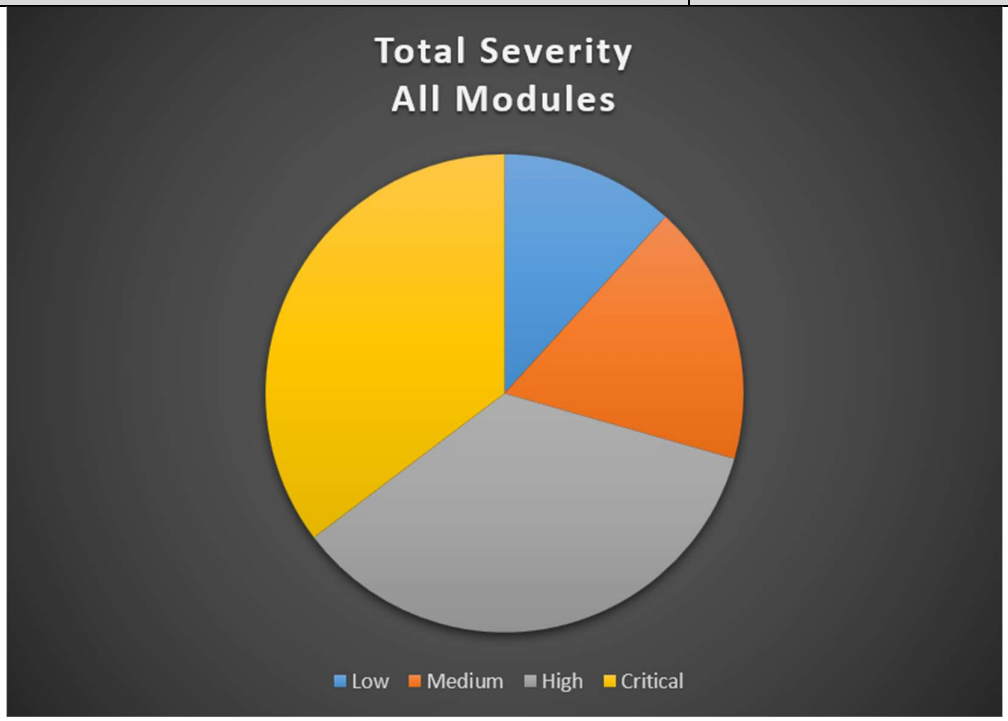
Moq was implemented to mock the applications audit service because it was of utmost importance to confirm that the program actually performed system interactions by

logging every activity that occurred on the program. By using mocks instead of stubs, it allowed me to assert that the interaction with the data layer actually occurred.

Defect Summary:

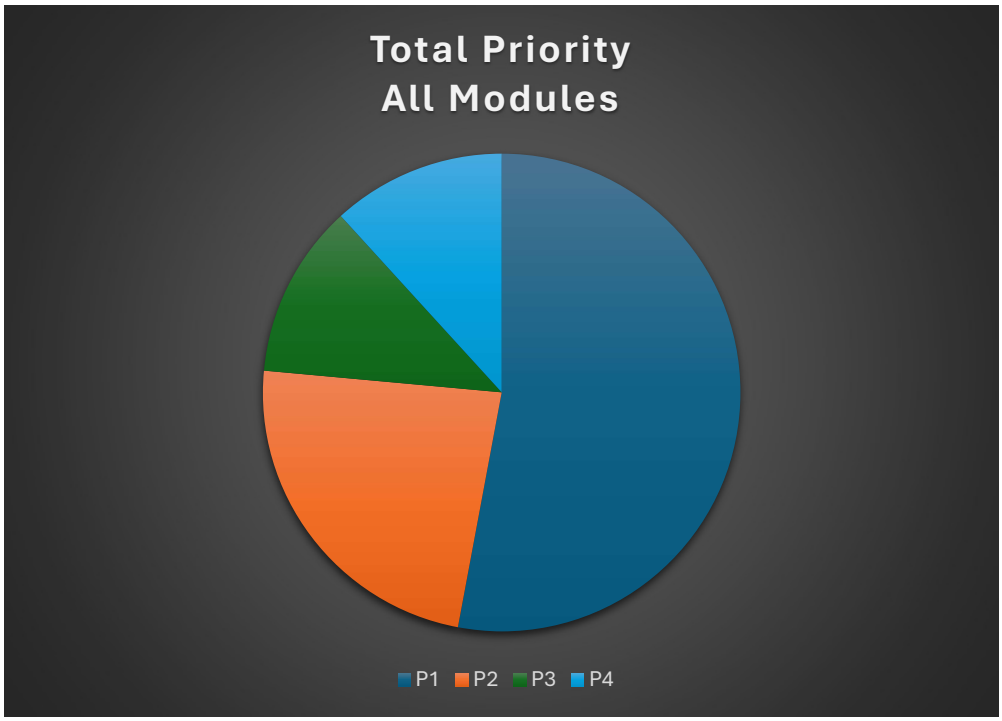
- Defect counts by severity:

Severity	Score
Low	2
Medium	3
High	6
Critical	6
Total	17



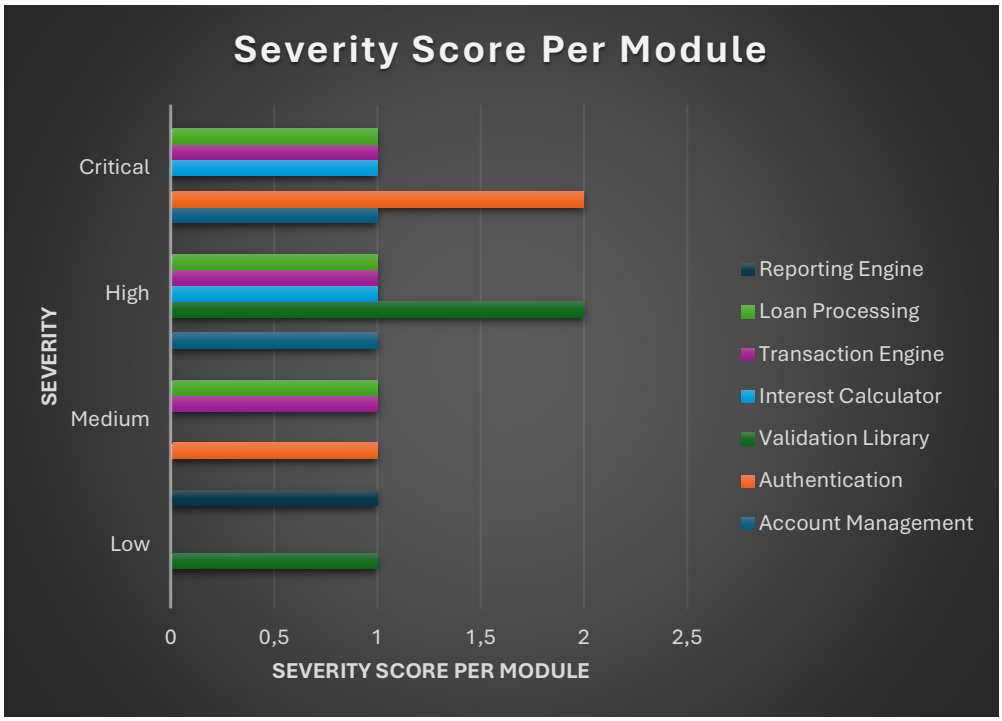
- Defect counts by priority:

Priority	Score
P1	9
P2	4
P3	2
P4	2
Total	17



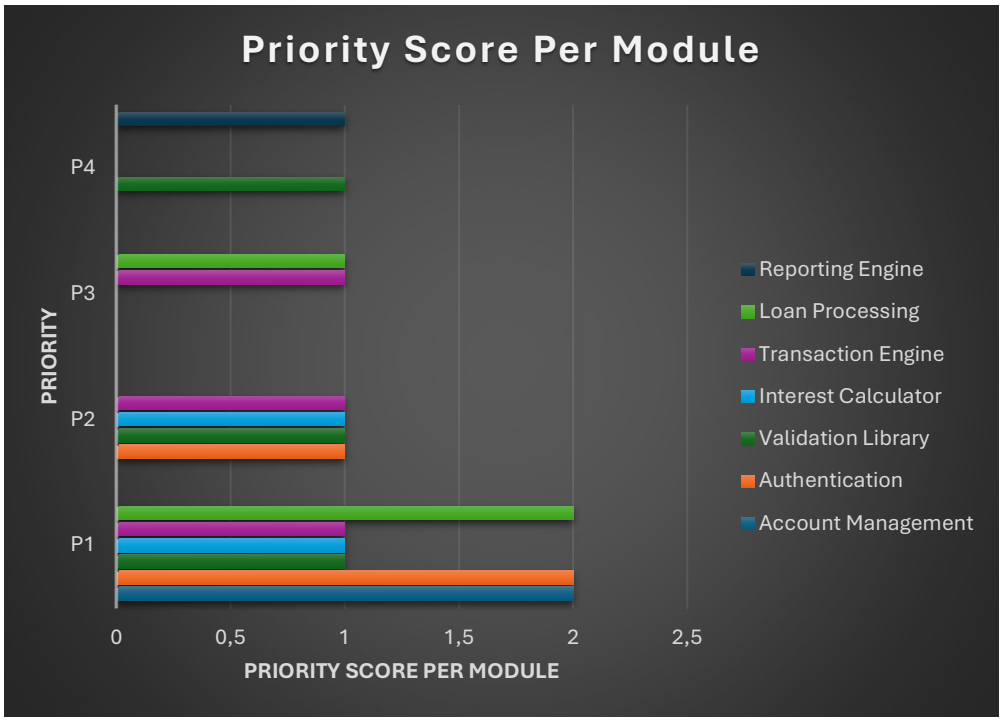
- Defect counts by severity per module:

Severity Per Module	Low	Medium	High	Critical
Account Management	0	0	1	1
Authentication	0	1	0	2
Validation Library	1	0	2	0
Interest Calculator	0	0	1	1
Transaction Engine	0	1	1	1
Loan Processing	0	1	1	1
Reporting Engine	1	0	0	0



- Defect counts by priority per module:

Priority Per Module	P1	P2	P3	P4
Account Management	2	0	0	0
Authentication	2	1	0	0
Validation Library	1	1	0	1
Interest Calculator	1	1	0	0
Transaction Engine	1	1	1	0
Loan Processing	2	0	1	0
Reporting Engine	0	0	0	1

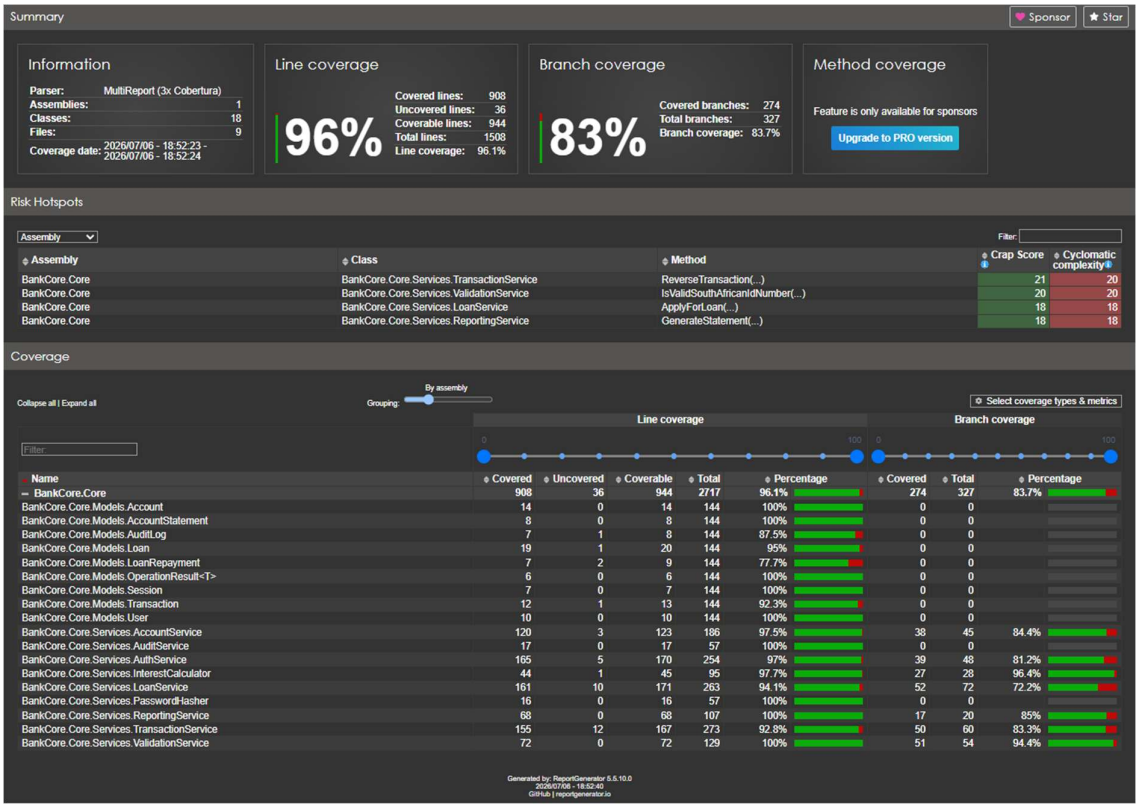


Defect Analysis:

Upon reviewing, most of the defects in the:

- Account Management Module was caused by assignment and logic errors
- Authentication Module was caused by conditions that were never checked
- Interest Calculator Module was caused by wrong formulas being used
- Loan Processing Module was caused by faulty formulas being used
- Reporting Engine Module was caused by not running validation checks on dates
- Transaction Engine Module was caused by incorrect conditional checks
- Validation Module was caused by faulty conditional checks

Coverage Report:



Risk Assessment Update:

Risk ID	Risk Description	Category	Likelihood (H/M/L)	Impact (H/M/L)	Risk Rating	Mitigation Strategy	Actual Results	Owner
R001	Financial calculation errors in interest module	Functional	High	High	Critical	Boundary value tests + formula verification	Formula errors where found	Tester
R002	Session token not invalidated on logout	Security	Medium	High	High	Auth test cases + manual trace	Logged out sessions still pass validation	Tester
R003	Able to transfer money to an account that does not exist, or one	Functional	Low	High	Medium	Design test cases that verify account status checks	Program does not allow funds to be transferred to non-	Tester

	which is closed						existant accounts	
R004	Negative values are accepted when making a deposit	Functional	Medium	High	High	Ensure to make use of equivalence partitioning to test negative input partitions	No negative values are accepted	Tester
R005	Application throws exceptions that are not handled when null or empty fields are detected	Functional	Medium	Medium	Medium	Check all fields for empty, null, or whitespace fields.	Application throws error messages when invalid data types are detected	Tester
R006	Tellers are able to access admin-role only functions	Security	Medium	High	High	Implement role-based access control. Test RBAC for all user types	Strong role based access conditions are inforced	Tester
R007	1000 sequential transactions exceed acceptable amount of time to process	Performance	Medium	Medium	High	Use the C# builtin stopwatch feature when running test scripts to measure execution time	Processing times are acceptable	Tester
R008	System freezes when generating a 12-month account statement for an account with 5000+ transactions	Performance	High	Medium	Medium	Use Nunit testing and implement the [Timeout] attribute to enforce performance SLA's	Program does not freeze	Tester
R009	Passwords are stored as plain text in	Security	Low	High	High	Test password hashing enforcement.	Passwords are hashed and salted	Tester

	program code							
R010	SQL injection strings and/or special characters crash the program	Robustness	High	High	Critical	Upon testing, ensure to insert known strings that are malicious as input parameters.	Program does not crash, or perform malicious actions when SQL injection strings are entered	Tester

Test Conclusion:

After the test phase has concluded, which has successfully highlighted several business-critical bugs in the financial formulas and conditional statements of the program logic.

After the identified bugs have been corrected (fixed as simulated), both the manual and automated tests have been rerun to eliminate any possibility of regression. Consequently, the BankCore Enterprise Management System is now considered as highly stable, and is ready to be deployed for use. Once the application is deployed, it is highly advisable that the application be monitored to quickly remediate any bugs in the very unlikely event that one occurs.

Recommendations:

Upon the successful completion of the testing phase, the following recommendations can be made:

1. It is recommended that the development team double check financial formulas when creating a financial application
2. It is also recommended that the development team ensure to use the correct conditional statements to ensure that errors don't occur at boundaries
3. It is recommended that the development team integrate third-party multi factor authentication to ensure that the application remains as secure as possible.
4. It is recommended that the development team perform a comprehensive code audit to ensure that the application remains robust and secure against any SQL injection strings.
5. It is highly recommended that the development team makes use of stricter session token management policies to ensure that session tokens are invalidated when a user is logged out

6. It is recommended that the development team implement CI/CD Integration so that automated tests are run on code every time it gets pushed to the application's repository.

Lessons Learned:

There are multiple lessons I have learned throughout the course of this testing phase, like how important documentation is on the program logic, as well as how important testing documentation is when it comes to a multiple phase testing phase.

There are a few things I would do differently when it comes to testing an application, like:

- Making sure that you should always remember to push your code to an online repository from the start to prevent data loss.
- I have also learned to make sure that you should always check whether the correct versions of packages are installed to prevent any runtime errors.
- Another huge lesson I have learned is that the quality of documentation is important, and should not be seen as an easy task.

Appendices:

PMT Formula Research:

After research, the formula below is the PMT formula is as follows:

PMT = (PresentValue x rate) / (1 – (1 + rate)^-n

It is derived from the PresentValue (PV) of the ordinary annuity formula:

Step 1: PV = PMT / (1 + r)^1 + PMT / (1 + r)^2 + PMT / (1 + r)^3

Step 2: PV = PMT[1 / (1 + r)^1 + 1 / (1 + r)^2 + 1 / (1 + r)^3]

Step 3: PV = PMT[1/1+r x 1 – (1/1+r)^n / 1 – 1/(1/1+r)]

Step 4: PV = PMT[1 – (1 + r)^-n / r]

Step 5: PMT = (PV x r) / (1 – (1+r)^-n)

Appendix A: Test Plan

File: TestPlan.docx

Appendix B: Test Cases

File: TestCases.xlsx

Appendix C: Test Execution Results:

File: TestCases – Executed.xlsx

Appendix D: Traceability Matrix:

File: TraceabilityMatrix.xlsx

Appendix F: Defect Log

File: DefectLog.xlsx

Appendix G: Risk Register:

File: RiskRegister.xlsx